

RAPPORT GLOBAL DES TRAVAUX PRATIQUES

Module	Big Data Technologies
Master	MLDS
Auteur	C21838
Année Universitaire	2025-2026

Introduction

Le module Big Data Technologies a pour objectif d'étudier les architectures distribuées permettant le stockage et le traitement de grands volumes de données.

Dans le cadre de ces travaux pratiques, nous avons mis en place un cluster Hadoop complet à l'aide de Docker puis exploré deux composantes fondamentales de l'écosystème Hadoop :

- HDFS (Hadoop Distributed File System)
- MapReduce (Framework de calcul distribué)

L'ensemble des expérimentations a été réalisé sur un cluster composé d'un NameNode et de deux DataNodes.

TP1 : Administration HDFS et Gestion du Stockage Distribué

Objectifs

- Comprendre l'architecture HDFS.
- Déployer un cluster Hadoop.
- Manipuler les fichiers distribués.
- Étudier la réplication des données.
- Observer le découpage des fichiers en blocs.
- Tester la tolérance aux pannes.

Architecture du Cluster

Composant	Fonction
NameNode	Gestion des métadonnées
DataNode1	Stockage des blocs
DataNode2	Stockage des blocs
Docker Network	Communication inter-nœuds

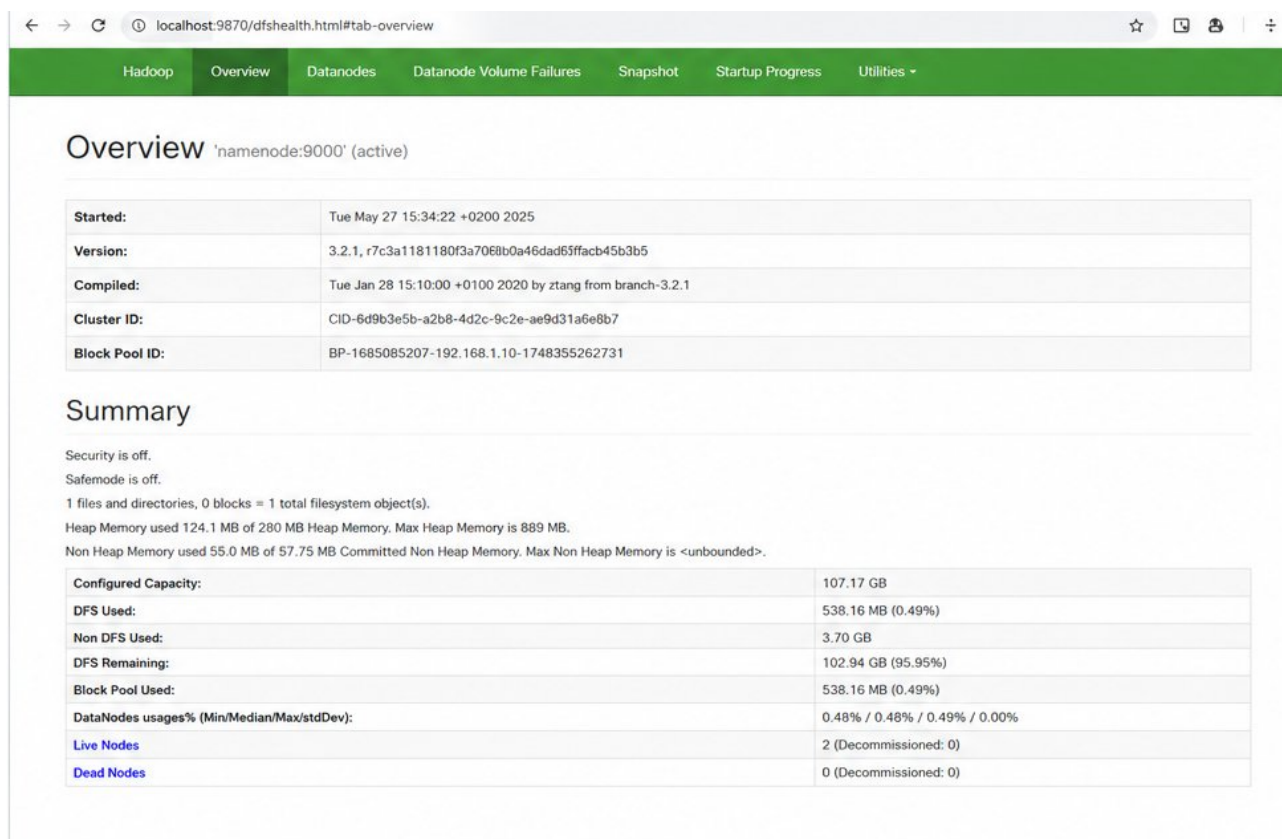
Démarrage du Cluster

```
docker compose up -d
docker ps
```

Les trois conteneurs principaux sont opérationnels : namenode, datanode1, datanode2.

Interface Web du NameNode

L'interface d'administration HDFS est accessible via **<http://localhost:9870>**. Elle permet la surveillance du cluster, la consultation des DataNodes, l'exploration de l'arborescence HDFS, la visualisation des blocs et le suivi de l'espace disque.



2. Interface NameNode – Overview (localhost:9870)

Figure 2. Interface NameNode – Overview (localhost:9870)

Création de l'Arborescence HDFS

```
hdfs dfs -mkdir -p /user/elkheit/data/parquet
hdfs dfs -ls /user/elkheit
```

Importation des Données

Un premier test a été réalisé avec **yellow_tripdata_2023-01.parquet**. Ce fichier étant inférieur à la taille d'un bloc HDFS, il ne permettait pas d'observer la distribution des blocs. Un second fichier a donc été utilisé : **yellow_tripdata_2010-01.parquet** (taille : plus de 500 Mo).

```
hdfs dfs -put \
/tmp/yellow_tripdata_2010-01.parquet \
/user/elkheit/data/parquet/
```

Analyse des Blocs HDFS

```
hdfs fsck \
/user/elkheit/data/parquet/yellow_tripdata_2010-01.parquet \
-files -blocks -locations
```

Bloc	Taille
Bloc 1	128 Mo
Bloc 2	128 Mo
Bloc 3	128 Mo

Bloc 4	≈116 Mo
--------	---------

Réplication

```
hdfs dfs -stat '%r'
```

Le cluster est configuré avec un **facteur de réplication égal à 2**. Chaque bloc est stocké sur plusieurs DataNodes afin d'assurer la disponibilité des données, la tolérance aux pannes et la haute disponibilité.

Tolérance aux Pannes

```
docker stop datanode1  
# ... données toujours accessibles  
docker start datanode1
```

Malgré l'arrêt d'un DataNode, les données restent accessibles grâce à la réplication. Après redémarrage, le cluster retrouve son fonctionnement normal.

TP2 : Maîtrise du Paradigme MapReduce

Objectifs

- Comprendre le paradigme MapReduce.
- Étudier le Shuffle.
- Comprendre les Combiners.
- Réaliser des jointures distribuées.
- Manipuler les graphes sociaux.
- Exécuter des jobs Hadoop sur le cluster.

Exercice 1 : Analyse de Logs

Format : **DATE | IP | URL | STATUS | SIZE**

Objectif : extraire uniquement les erreurs HTTP 404. Le Mapper émet **404** → **1**. Le Reducer est facultatif (filtrage uniquement). Pour compter les requêtes par IP, l'adresse IP devient la clé du Mapper.

Exercice 2 : Inverted Index

Objectif : construire un moteur de recherche simplifié. Le Reducer élimine les doublons et construit l'index inversé.

Entrée :

docA : Hadoop Spark
docB : Hadoop Docker

Sortie :

Hadoop -> [docA, docB]
Spark -> [docA]
Docker -> [docB]

Exercice 3 : Friends in Common

Entrée : A -> B, C, D

Mapper :

(B,C) -> A
(B,D) -> A
(C,D) -> A

Reducer :

(B,C) -> [A]
(B,D) -> [A]
(C,D) -> [A]

Complexité : $O(n^2)$

Pour une célébrité possédant plusieurs millions d'amis, l'étape de Shuffle devient extrêmement coûteuse.

Exercice 4 : Optimisation via le Combiner

Calcul de la température moyenne. Une moyenne n'est pas directement combinable. Le Combiner transmet (**ville, somme, nombre**) — ex : **Nouakchott** → **(88, 3)**. Le Reducer calcule ensuite la moyenne finale, réduisant fortement le trafic réseau.

Exercice 5 : Jointures Distribuées

Reduce-Side Join

Toutes les données transitent par le Shuffle. **Avantages** : flexible, applicable à tout type de données. **Inconvénient** : trafic réseau important.

Map-Side Join

La petite table Produits est chargée en mémoire via le **Distributed Cache**. **Avantages** : pas de Shuffle, exécution plus rapide, réduction du trafic réseau.

Travail Pratique sur Cluster

Implémentation d'un job Hadoop Streaming pour compter les erreurs HTTP :

404 -> N

500 -> M

403 -> P

Interface YARN

Accessible à **http://localhost:8088**, elle permet le suivi des applications, la surveillance des tâches Map et Reduce, et l'analyse des Counters Hadoop.

Analyse des Counters

Les compteurs Hadoop mesurent les données lues/écrites, les enregistrements traités et les octets transférés durant le Shuffle. Ces indicateurs sont essentiels pour l'optimisation des performances.

Automatisation des TP

HDFS Manager

- Upload de fichiers
- Gestion des permissions et de la réplication
- Analyse des blocs
- Utilisation disque
- Safe Mode / gestion des DataNodes

MapReduce Manager

- WordCount
- Analyse de logs / comptage d'erreurs HTTP
- Inverted Index
- Friends in Common
- Jointures distribuées
- Monitoring YARN / analyse des Counters

BigDataManager

Gestionnaire global centralisant l'ensemble des fonctionnalités du cluster. Modules disponibles :

- HDFS
- MapReduce
- YARN
- Spark
- Hive
- Monitoring
- Génération de rapports

Conclusion Générale

Ces travaux pratiques ont permis de maîtriser les principaux composants de l'écosystème Hadoop.

Le **TP HDFS** a mis en évidence les mécanismes de stockage distribué, de réplication et de tolérance aux pannes.

Le **TP MapReduce** a permis de comprendre la décomposition d'un traitement en phases Map, Shuffle et Reduce, ainsi que les problématiques d'optimisation du calcul distribué.

L'ensemble des expérimentations a confirmé l'efficacité des architectures Big Data pour le traitement de volumes importants de données et a permis la mise en œuvre concrète des concepts étudiés dans le module **Big Data Technologies**.